

Sự Tiến Hóa Của D3QN (Batch 1: Nền Tảng & Vết Xe Đổ)

1. Kỷ Nguyên Đồ Đá: Q-Learning Cổ Điển (Bảng Excel chạy bằng cơm)

Cội nguồn của mọi thuật toán Value-based RL đều bắt đầu từ **Q-Learning**. Tưởng tượng mày có một cái ma trận 2 chiều (Q-Table):

- **Hàng:** Đại diện cho tất cả các Trạng thái có thể xảy ra (s).
- **Cột:** Đại diện cho tất cả các Hành động có thể thực hiện (a).

Ô giao nhau chứa giá trị **Q-Value**, biểu thị độ "ngon" của việc thực hiện hành động a khi đang ở trạng thái s . Bot cứ đi loanh quanh, chọn hành động, nhận điểm thưởng (r), rồi quay lại cập nhật cái bảng đó theo phương trình Bellman:

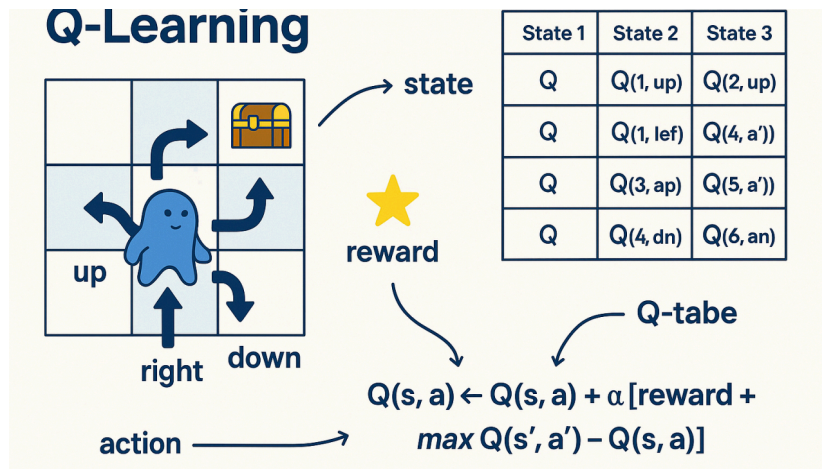
$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

Dịch ra ngôn ngữ loài người: Giá trị mới = Giá trị cũ + Tốc độ học $\times$ (Thực tế + Kỳ vọng tương lai - Dự đoán cũ).

Lỗi chí mạng (Curse of Dimensionality):

Trò này chỉ chơi được mấy cái map dạng lưới (Gridworld) hay mê cung bé tí. Khi mày vác vào game Atari, đầu vào là ảnh pixel. Mỗi pixel có 256 giá trị màu, kích thước ảnh là 84×84 , lại còn stack 4 frames.

Số lượng trạng thái có thể xảy ra lớn hơn số lượng nguyên tử trong vũ trụ. Mày cầm con H100 hay siêu máy tính lượng tử cũng đéo có đủ RAM để chứa nổi cái Q-Table này. Bế tắc toàn tập.



2. Gắn Não Nhân Tạo: Deep Q-Network (Vanilla DQN - 2013)

Năm 2013, bọn DeepMind nảy ra một ý tưởng thay đổi cuộc chơi: **Đập bỏ cái bảng Q-Table đi**, dùng **Mạng Nơ-ron Tích chập (CNN)** làm hàm xấp xỉ.

Đầu vào không phải là tọa độ nữa, mà là ảnh raw của màn hình game. Đầu ra là một mảng dự đoán Q-Value cho toàn bộ các nút bấm trên tay cầm. Thay vì dò bảng và cập nhật từng ô, mày dùng Backpropagation (Lan truyền ngược) để cập nhật trọng số (weights θ) của mạng nơ-ron thông qua hàm Loss:

$$L(\theta) = \mathbb{E} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right]$$

1. Ký hiệu Kỳ vọng: $\mathbb{E}[\dots]$

Trong toán học, nó là Expected Value (Giá trị kỳ vọng). Nhưng khi mày code cái file `agent.py`, nó chính là việc mày lấy trung bình (mean) của hàm Loss trên một cái **Mini-batch** (thường là 32 mẫu (s, a, r, s')) bốc ngẫu nhiên ra từ Replay Buffer. Mày không học trên từng frame một vì nó sẽ làm gradient bị giật cục (noisy gradients), học trên một mẻ (batch) sẽ giúp model ổn định hơn.

2. Sự ảo tưởng hiện tại (Prediction): $Q(s, a; \theta)$

Đây là câu trả lời của mạng nơ-ron Online (với bộ trọng số θ đang liên tục được cập nhật) ở bước hiện tại: "Ê, tao đang ở trạng thái s , nếu tao làm hành động a , tao nghĩ tao sẽ được chừng này điểm."

3. Chân lý mục tiêu (Target): $r + \gamma \max_{a'} Q(s', a'; \theta^-)$

Đây là cái mốc chuẩn mực mà máy ép cái mạng Online phải học theo. Nó gồm 2 phần cộng lại:

- r (Immediate Reward): Tiền tươi thóc thật. Điểm số máy nhận được ngay lập tức sau khi thực hiện hành động a .
- $\gamma \max_{a'} Q(s', a'; \theta^-)$ (Discounted Future Value): Đây là cái tầm nhìn xa. Từ trạng thái mới s' , máy dùng mạng Target (cái mạng già làng θ^-) để ngó xem hành động a' nào là ngon ăn nhất ở tương lai, rồi lấy cái đỉnh $\max$ đó.
- γ (Discount Factor): Nằm trong khoảng $[0, 1]$. Đặt gần 1 (ví dụ 0.99) nghĩa là máy dạy con bot biết lo xa (phần thưởng tương lai quan trọng ngang ngửa phần thưởng hiện tại). Đặt bằng 0 nghĩa là con bot bị thiếu cận, chỉ biết đớp cái r trước mắt.

4. Đánh giá độ ngu (Mean Squared Error): $(\dots)^2$

Mày lấy Chân lý (Target) - Dự đoán (Prediction). Nếu ra 0, nghĩa là mạng nơ-ron đoán như thần, đéo cần sửa gì cả (Loss = 0). Nếu ra khác 0, máy bình phương nó lên (tránh dấu âm và khuếch đại lỗi lớn). Cái sai số này chính là mũi nhọn để thuật toán Backpropagation đạo hàm ngược lại, ép trọng số θ của mạng Online tự uốn nắn sao cho lần sau nó dự đoán sát với Chân lý hơn.

Nhìn thì có vẻ hoàn hảo, DeepMind còn lên hẳn trang bìa tạp chí Nature vì cái trò này. Nhưng dùng một thời gian, dân tình phát hiện ra một cái lỗ hổng vỡ mặt.

Lỗi chí mạng (Overestimation Bias):

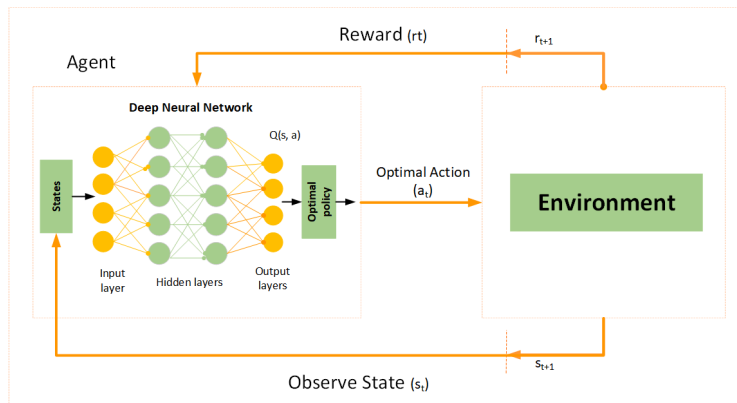
Mày nhìn kỹ vào cái cụm Target dùng để dạy model: $r + \gamma \max_{a'} Q(s', a'; \theta^-)$.

Thằng DQN Vanilla nó dùng cùng một mạng nơ-ron để kiểm luôn 2 việc:

1. **Lựa chọn:** Đi tìm hành động có giá trị Q cao nhất ($\arg \max$).
2. **Đánh giá:** Tính luôn xem cái giá trị Q đó cụ thể là bao nhiêu điểm.

Vấn đề: Mạng nơ-ron lúc đầu toàn là rác (noise). Sẽ có những hành động ngu học nhưng vô tình bị model gán cho một giá trị Q vống lên tận trời do nhiễu. Hàm $\max$ nó cực kỳ tham lam, nó chộp ngay lấy cái con số ảo tưởng đó làm mục tiêu để học.

Hệ quả? Model của mày biến thành một thằng ảo tưởng sức mạnh (Overoptimistic). Giá trị Q bị bơm thổi liên tục đến mức nổ tung, bot chạy vòng vòng làm trò con bò vì tưởng thể là hay. Mạng sập.



Sự Tiến Hóa Của D3QN (Batch 2: Liều Thuốc Giải & Tiến Hóa Hoàn Hảo)

3. Thuốc Giải 1: Double DQN (DDQN - 2015) - Trị Bệnh Ảo Tưởng

Để chữa cái bệnh Overestimation Bias (ảo tưởng sức mạnh) của Vanilla DQN tao nói ở Batch 1, DeepMind nhận ra vấn đề nằm ở việc vừa đá bóng vừa thổi còi. Mà không thể dùng 1 cái mạng nơ-ron để tự chọn hành động rồi tự chấm điểm cho hành động đó được.

Triết lý của DDQN: Tách đôi công việc ra cho 2 mạng độc lập:

1. **Mạng Online (θ):** Thằng trâu tre, cập nhật liên tục, chuyên đi "chọn hàng" (tìm hành động tốt nhất).
2. **Mạng Target (θ^-):** Thằng già làng, cập nhật chậm hơn (sau mỗi N steps), chuyên làm nhiệm vụ "thẩm định giá".

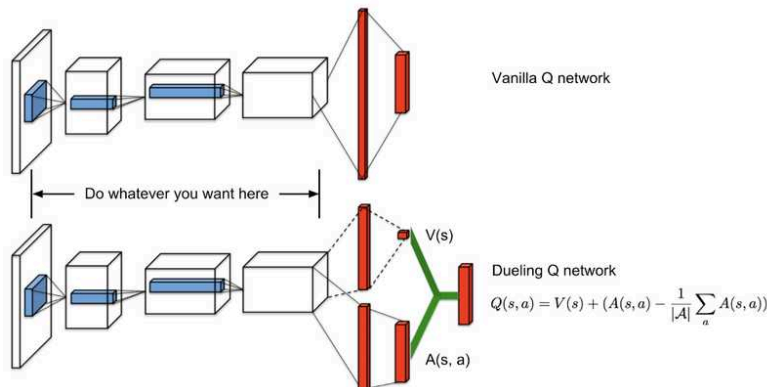
Công thức mục tiêu (Target) được sửa lại cực kỳ tinh tế:

$$Y_t^{DoubleDQN} = r + \gamma Q(s', \arg \max_{a'} Q(s', a'; \theta); \theta^-)$$

$$Y = r + \underbrace{\gamma Q(s', \arg \max_{a'} Q(s', a'; \theta); \theta^-)}_{\text{Lão Sếp định giá}}$$

Nó hoạt động thế nào?

- Thằng Online θ nhảy ra bảo: "Ê tao thấy hành động a' này ngon nhất nè ($\arg \max$)."
- Thằng Target θ^- rút bảng điểm ra soi: "Đề tao đánh giá lại xem mày chọn chuẩn không. Mày chọn ngu do nhiễu (noise) thì tao cho điểm thấp."
- Nhờ cơ chế chéo cánh này, một hành động rác rưởi lỡ bị thằng Online đánh giá cao vống lên sẽ ngay lập tức bị thằng Target dội gáo nước lạnh. Giá trị Q được kim hãm về đúng thực tế. Bệnh ảo tưởng được trị tận gốc.



4. Thuốc Giải 2: Dueling Architecture (2016) - Phân Bỏ Tài Nguyên Não Bộ

DDQN đã fix được lỗi tính toán toán học, nhưng thiết kế mạng CNN lúc đó vẫn ngu ở chỗ bắt model học trực tiếp giá trị $Q(s, a)$ cho mọi trường hợp.

Triết lý Dueling: Trong game Atari (hay đời thực cũng vậy), có những lúc mày làm cái đéo gì cũng vô dụng. Ví dụ: Quả bóng trong game Pong đã bay lọt qua mép cái ván đỡ của mày rồi. Lúc này mày có bấm lên, xuống, hay đứng im thì kết quả vẫn là mất điểm (-1). Việc bắt mạng nơ-ron phải tốn tài nguyên đi tính toán chi tiết từng hành động trong tình huống "chắc chắn chết" là sự ngu dốt.

Mạng Dueling giải quyết bài toán này bằng cách chẻ đôi cái mạng CNN ra ở những lớp cuối cùng (Fully Connected layers) thành 2 luồng độc lập:

1. **State-Value function $V(s)$:** Đánh giá tổng quan xem bản thân Trạng thái (s) này là đồng rác hay mỏ vàng (không thèm quan tâm hành động).
2. **Advantage function $A(s, a)$:** Đánh giá xem nếu ở trạng thái này, hành động a mang lại lợi thế cao hơn hay thấp hơn mức trung bình bao nhiêu.

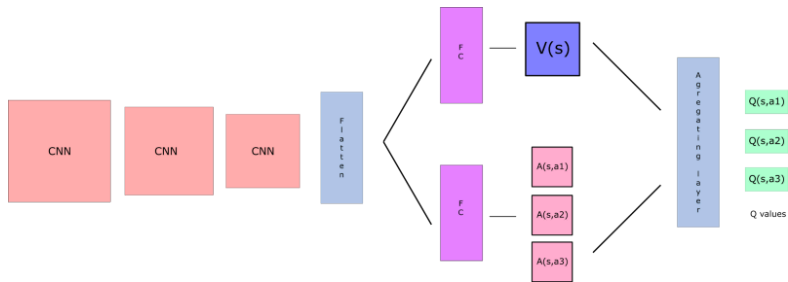
Cuối cùng, nó gộp 2 luồng này lại để tạo ra $Q(s, a)$ cuối cùng bằng công thức:

$$Q(s, a; \theta, \alpha, \beta) = V(s; \theta, \beta) + \left(A(s, a; \theta, \alpha) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a'; \theta, \alpha) \right)$$

Tại sao lại có cái cụm trừ đi trung bình $\frac{1}{|\mathcal{A}|} \sum_{a'}$ kia?

Đây là tính **Identifiability** (Khả năng nhận dạng). Nếu mà chỉ cộng đơn thuần $V + A$, model có thể lách luật bằng cách cộng bù 10 điểm vào V và trừ đi 10 điểm ở A , kết quả Q vẫn vậy nhưng ý nghĩa của V và A bị nát. Việc trừ đi trung bình ép mạng nơ-ron phải học đúng: $V(s)$ phải là giá trị gốc rễ, còn $A(s, a)$ chỉ là phần dao động xoay quanh gốc đó.

Nhờ kiến trúc Dueling, con bot học cực kỳ nhanh. Nó biết phân bổ sự tập trung: Lúc nào bóng tới gần thì dồn sức tính toán hành động (quan tâm Advantage), lúc bóng ở xa tít mù tắp thì thư giãn kệ mẹ sự đời (chỉ quan tâm Value).



Sự Tiến Hóa Của D3QN (Batch 3: Vũ Khí Bí Mật & Manh Ghép Cuối)

6. Mắt Thần Mù Dở: Tại sao phải dùng Frame Stacking?

Trước khi ném data vào mạng D3QN, mày phải xử lý cái màn hình game Atari đã.

- Vấn đề của 1 Frame:** Nếu tao đưa cho mày 1 tấm ảnh chụp màn hình game Pong, mày sẽ thấy 2 cái ván và 1 quả bóng đứng im. Mày đéo thể nào biết quả bóng đang bay sang trái hay sang phải, bay nhanh hay bay chậm. Trạng thái (s) bị thiếu thông tin trầm trọng (phá vỡ tính Markov). Model sẽ đoán mò và nát bét.
- Cách giải quyết:**
 - Chuyển ảnh xám & Thu nhỏ:** Game Pong màu mè lấp lánh đéo có ý nghĩa mẹ gì. Chuyển mẹ hết về ảnh xám (Grayscale) và crop/resize xuống còn 84×84 pixel để tiết kiệm RAM.
 - Frame Stacking (Chồng ảnh):** Gom 4 frames liên tiếp lại thành 1 block (kích thước $4 \times 84 \times 84$). Nhờ có 4 khung hình liên tục, mạng CNN (qua các bộ lọc Conv2D) sẽ tự động nội suy ra được Vận tốc và Hướng đi của quả bóng. Đây là bước sống còn để định hình Trạng thái (State) trong Atari.

7. Ôn Thi Kiểu Thiên Tài: Prioritized Experience Replay (PER)

Hồi này tao bảo con bot đi chơi game, nhét kinh nghiệm (s, a, r, s') vào một cái bộ nhớ (Replay Buffer) rồi bốc random ra học. Chơi kiểu random đó (Vanilla Replay) cực kỳ ngu và tốn thời gian.

Triết lý của PER (2015): Mày đi học, bài nào dễ mày giải 1 lần là nhớ. Bài nào khó, mày giải sai (điểm kém), mày phải lòi ra ôn lại 10 lần. Con AI cũng vậy. Có những khung hình trong game chả có mẹ gì xảy ra (hài bên đứng nhìn bóng trôi), học mớ data đó là tốn điện con H100. Nó phải ưu tiên học những pha bị thủng lưới, hoặc những pha cứu bóng vớt vát (những pha làm nó "bất ngờ" nhất).

Toán học đẳng sau PER:

Độ "bất ngờ" của model được đo bằng **TD Error** (δ). TD Error càng cao nghĩa là dự đoán của con AI lúc đó càng ngu lệch so với thực tế.

$$\delta_i = r + \gamma Q(s', \arg \max_{a'} Q(s', a'; \theta); \theta^-) - Q(s, a; \theta)$$

Xác suất lấy mẫu một trải nghiệm i ra học không còn là chia đều, mà tỷ lệ thuận với cái độ ngu đó:

$$P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha}$$

(Trong đó $p_i = |\delta_i| + \epsilon$ để đảm bảo trải nghiệm nào cũng có tí cơ hội được bốc trúng, và α kiểm soát mức độ mày muốn "thiên vị" những trải nghiệm điểm cao).

Tuy nhiên, vì mày bốc mẫu có chủ đích (bias), mày đang thay đổi phân phối gốc của dữ liệu. Nếu không sửa lại, model sẽ bị cập nhật sai lệch. Nên PER bắt buộc phải dùng **Importance Sampling (IS) weights** để bù trừ trong lúc tính Loss lúc Backprop:

$$w_i = \left(\frac{1}{N \cdot P(i)} \right)^\beta$$

Mọi thứ chạy bằng cấu trúc dữ liệu **SumTree** $O(\log N)$ nên tốc độ bốc mẫu nhanh như chớp, ép con bot phải nhìn thẳng vào sự thật ngu dốt của nó để tiến bộ nhanh gấp đôi.

8. Bức Tranh Tổng Thể (The Grand Pipeline)

Tóm gọn lại, cái đồng HW4 mày đang chạy trên server là sự kết hợp của 4 tầng công nghệ, ráp lại thành cỗ máy xay chả D3QN:

- Vision (Tiền xử lý):** Mắt thu nhận 4 frames liên tiếp (84×84) ép thành 1 State hoàn chỉnh.
- Memory (PER):** Ghi nhớ mọi hành động, nhưng chỉ ưu tiên bốc những pha "nhớ đời" (TD Error cao) ra để học lại.
- Brain Architecture (Dueling Network):** CNN tách não làm 2 luồng $V(s)$ và $A(s, a)$ để biết khi nào cần tập trung hành động, khi nào kệ mẹ sự đời.
- Learning Logic (Double DQN):** Hai anh em Online Network và Target Network giám sát chéo nhau, thằng Online chọn hành động, thằng Target chấm điểm, diệt trừ vĩnh viễn bệnh Ảo tưởng sức mạnh (Overestimation).

